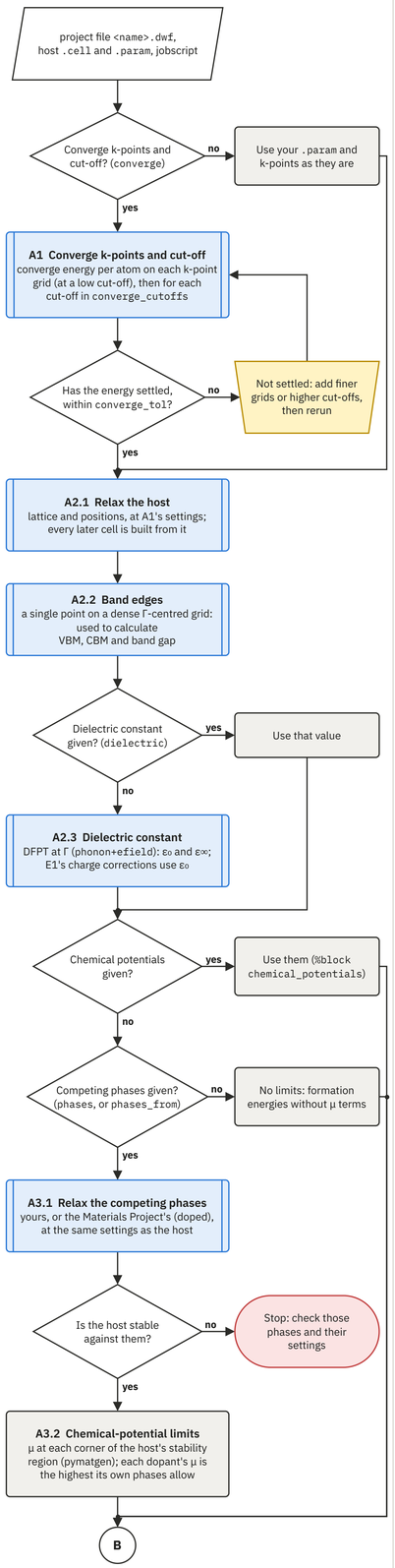
Defect workflow, as dwf.py runs it

Read each column from top to bottom, A to F. Diamonds are decisions, and every branch out of one is labelled; "given" means set in the project file. One command runs every phase in order: `python3 dwf.py <name>.dwf`. Each phase writes summary.txt (for you to read) and results.json (for the later phases) in its own folder.

start / end · done by dwf.py itself · CASTEP jobs (run through supercell_convergence.py) · MACE relaxations (charge-blind; in dwf.py, or as jobs on compute nodes) · ShakeNBreak · needs your attention · decision · input · (B) continues in the next column

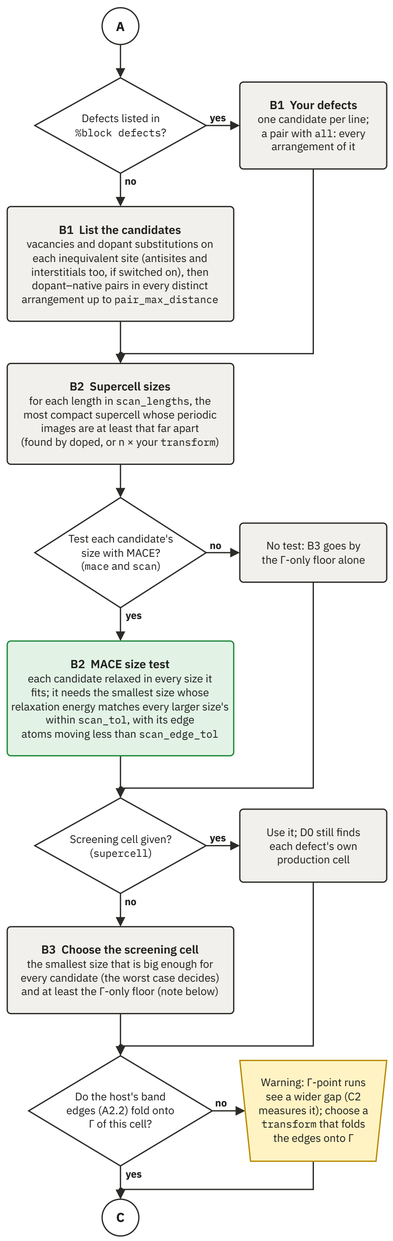
A Host settings and reference data

A_host/ · once per host



B Candidate defects and the screening cell

B_candidates/ · once per host



The Γ-only floor (B3)

C screens at the Γ point only, which gives a defect's levels as an isolated defect has them only when its periodic images barely overlap. A level E_b inside the gap decays as $\exp(-r/\xi)$, where (two-band model) $1/\xi^2 = (2m^*/\hbar^2)\,E_b\,(1 - E_b/E_g)$. The screening cell keeps its images at least 4ξ apart, with

- E_b = `deep_level_margin`: the shallowest level that counts as deep
- m^*: the lighter of the hole and electron masses at A2.2's band edges
- E_g: A2.2's band gap

Set `screen_min_image` to give the floor yourself (0: no floor).

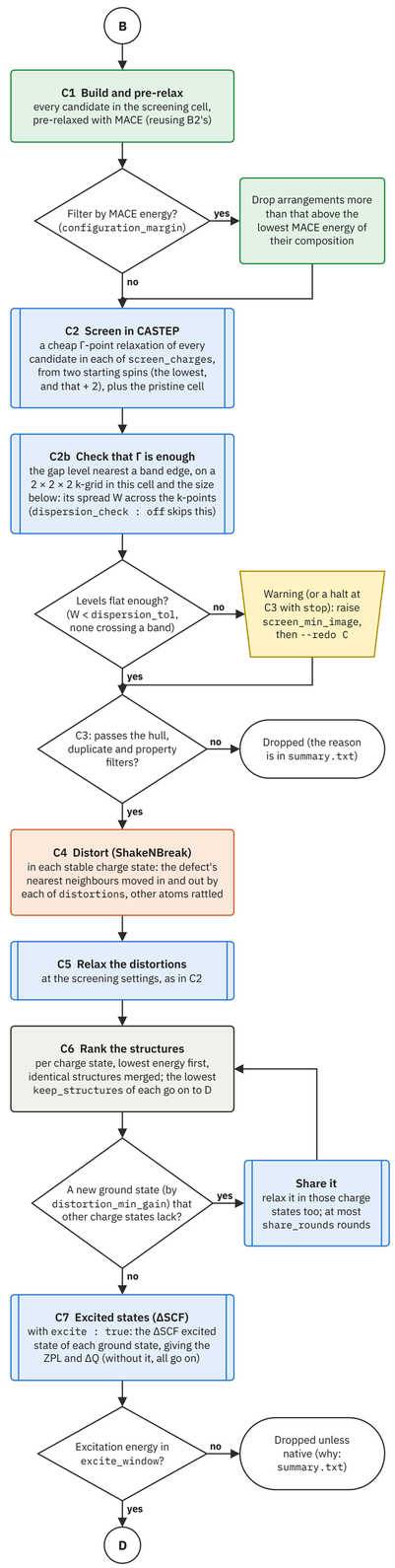
Cell sizes are image distances

A size is the shortest distance between a defect and its periodic images (Å), not an atom count. The sizes come from `scan_lengths`:

- B2: MACE tests every candidate in every size
- B3: the screening cell fits the worst case and the Γ-only floor
- C2b: checks the screening cell with 2 × 2 × 2 k-points
- D0: steps up the sizes to each defect's own production cell

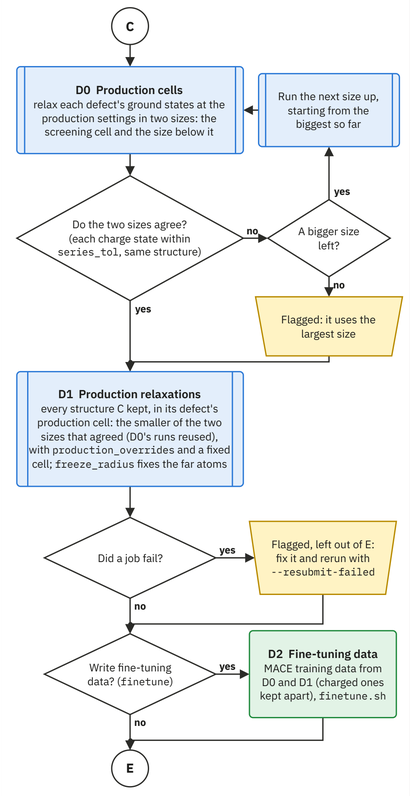
C Screening and ground states

C_screen/ · every candidate, then the survivors



D Production cells and relaxations

D_production/ · each surviving defect, in every charge state



How the CASTEP jobs run

Every CASTEP step hands its jobs to `supercell_convergence.py`:

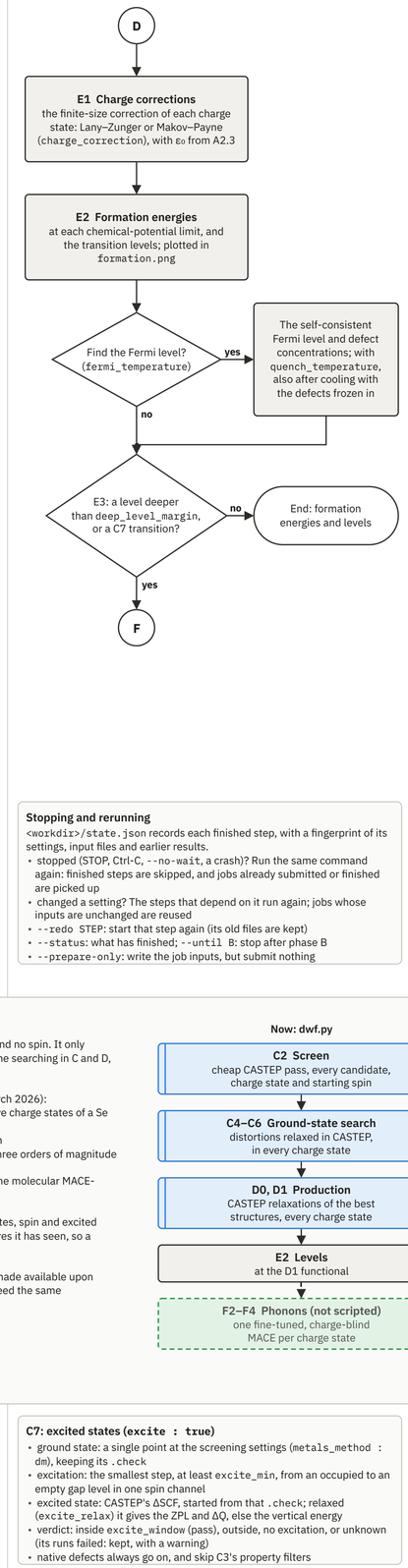
- one short dry run per distinct input sizes the jobs: nodes, memory, partition and walltime
- no more than `max_jobs` are queued at once
- a job that runs out of time carries on from its `.check` (up to `max_restarts` times; screening jobs don't restart)
- with `tolerate_failures` (screening, D0, D1), a failed job is flagged and the others carry on
- a finished job whose inputs have since changed stops the run, so results from two different settings never mix
- `touch <workdir>/STOP` stops dwf.py; queued jobs keep running

What the filters in C keep

- C1 `configuration_margin`: arrangements within that of the lowest MACE energy of their composition (MACE ignores the charge)
- C3 hull: an arrangement stays if, at some Fermi level in the gap, its lowest formation energy is within `hull_margin` of its group's
- C3 duplicates: one that relaxed into a lower one's structure goes
- C3 properties (optional), met by a charge state stable somewhere in the gap: `require_gap_states`, `require_spin`, `require_bound`
- C6: the `keep_structures` lowest structures of each charge state

E Thermodynamics

E_thermo/ · every defect that reached D



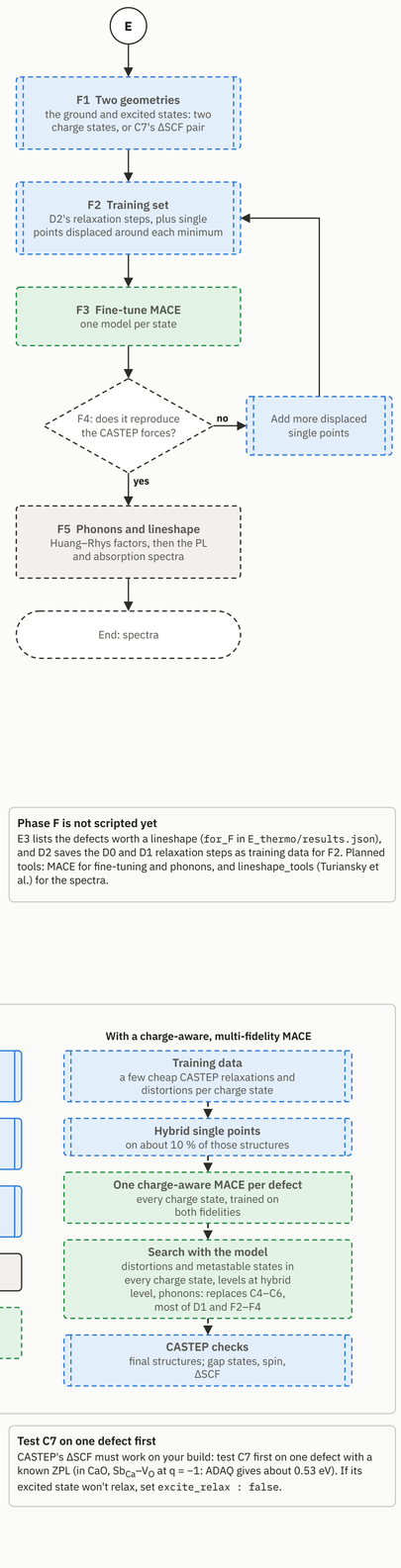
Stopping and rerunning

`<workdir>/state.json` records each finished step, with a fingerprint of its settings, input files and earlier results.

- stopped (STOP, Ctrl-C, `--no-wait`, a crash)? Run the same command again: finished steps are skipped, and jobs already submitted or finished are picked up
- changed a setting? The steps that depend on it run again; jobs whose inputs are unchanged are reused
- `--redo STEP`: start that step again (its old files are kept)
- `--status`: what has finished; `--until B`: stop after phase B
- `--prepare-only`: write the job inputs, but submit nothing

F Optical lineshape

not scripted yet · selected defects



Phase F is not scripted yet

E3 lists the defects worth a lineshape (`for_F` in `E_thermo/results.json`), and D2 saves the D0 and D1 relaxation steps as training data for F2. Planned tools: MACE for fine-tuning and phonons, and lineshape_tools (Turiansky et al.) for the spectra.

Charge-aware MACE: not in the workflow yet

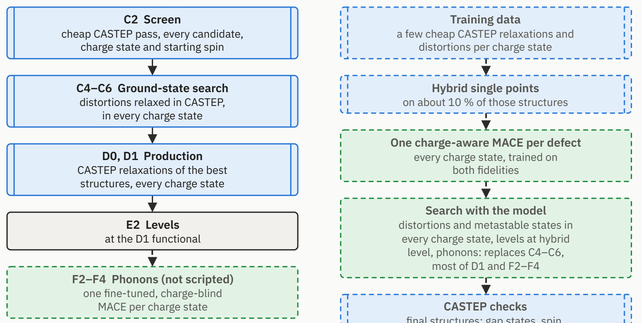
Every MACE step in this chart (green) is charge-blind: the model sees no charge and no spin. It only prepares structures (B2, C1) and never ranks charge states, so CASTEP does all the searching in C and D, and F needs one fine-tuned model per charge state.

A charge-aware MACE (Wang, Mosquera-Lois and Walsh, arXiv:2603.05238, March 2026):

- a learnable total-charge embedding in MACE (v0.3.15): one model covers all five charge states of a Se vacancy in Sb_2Se_3
- multi-fidelity: PBE relaxations, plus HSE06 single points on about 10 % of them
- transition levels within 0.02 eV of HSE06; exploring the energy surface about three orders of magnitude cheaper
- models with explicit charges (MACE-POLAR, MACE-LES) did worse there, and the molecular MACE-POLAR-1 missed the DFT ground states

What would stay. CASTEP makes the training data and has the last word: gap states, spin and excited states still need it. A model trained on one defect is reliable only near the structures it has seen, so a minimum it finds far from them is a candidate to check in CASTEP, not a result.

Status. One preprint and one test material; its data and trained weights "will be made available upon publication". It used VASP (PAW); with CASTEP, the PBE and hybrid runs would need the same pseudopotentials and settings.



C7: excited states (`excite : true`)

- ground state: a single point at the screening settings (`metals_method : dm`), keeping its `.check`
- excitation: the smallest step, at least `excite_min`, from an occupied to an empty gap level in one spin channel
- excited state: CASTEP's ΔSCF, started from that `.check`; relaxed (`excite_relax`) it gives the ZPL and ΔQ, else the vertical energy
- verdict: inside `excite_window` (pass), outside, no excitation, or unknown (its runs failed: kept, with a warning)
- native defects always go on, and skip C3's property filters

Test C7 on one defect first

CASTEP's ΔSCF must work on your build: test C7 first on one defect with a known ZPL (in CaO, Sb_{Ca}–V_O at q = −1: ADAQ gives about 0.53 eV). If its excited state won't relax, set `excite_relax : false`.